Concevoir et Choisir une Structure Répétitive Adaptée

Méthode algorithmique de choix, sémantique d'arrêt et implémentation C++ rigoureuse (while, do...while, for)

1. La Démarche de Conception : Quel Schéma Choisir ?

En algorithmique, plusieurs structures itératives permettent d'arriver au même résultat calculatoire. Cependant, **on ne choisit jamais une boucle par habitude ou par hasard** : on recherche la structure qui traduit le plus fidèlement la logique du problème, selon ce que l'on sait du nombre d'itérations et du moment d'évaluation des conditions d'arrêt (référence méthodologique : Patrick Etcheverry & Steve McConnell).

Situation Algorithmique	Structure C++ Naturelle	Moment d'évaluation	Nb Itérations
Nombre de répétitions connu ou calculable avant d'entrer dans la boucle	for (int i = 0; i < n; i++)	En-tête (compteur incrémenté)	Déterminé (N fois)
Condition d'arrêt testée avant le traitement (pré-condition)	while (!(condition_arret))	Au début (avant le corps)	**0 à N fois** (peut être sauté)
Condition d'arrêt testée après le traitement (post-condition)	do { ... } while (!(arret));	À la fin (après le corps)	**Au moins 1 fois** garanti
Condition d'arrêt évaluée en cours de répétition (entre Q1 et Q2)	while (true) { Q1; if (arret) break; Q2; }	Au milieu du traitement	Sortie anticipée break

2. Sémantique et Bonnes Pratiques C++ (Core Guidelines)

A. La Boucle Comptée : for (C++ Core Guidelines ES.72)

- **Usage impératif :** Dès qu'une variable de boucle ou un compteur évident existe (nombre de tours connu avant d'entrer).
- **Syntaxe unifiée :** for (initialisation; condition_maintien; incrementation) { ... }.
- **Règle McConnell :** Le contrôle de la boucle doit rester simple et confiné à l'en-tête (ne jamais altérer le compteur dans le corps).

B. La Boucle à Pré-condition : while (C++ Core Guidelines ES.73)

- **Usage impératif :** Lorsque l'arrêt dépend d'un événement externe (fin de flux, saisie d'une sentinelle) et que le traitement peut ne jamais s'exécuter si la condition est fausse dès l'entrée.
- **Forme canonique :** while (!condition_arret) (ou while (condition_maintien)).
- **Piège d'examen :** Oublier d'initialiser les variables de test avant le while ou oublier de modifier l'état dans le corps (boucle infinie).

C. La Boucle à Post-condition : do...while

- **Usage impératif :** Lorsque le traitement doit s'exécuter **au moins une fois** avant de décider de s'arrêter (cas canonique : saisie contrôlée au clavier avec redemande).
- **Syntaxe stricte :** do { ... } while (!condition_arret); — **Le point-virgule final est obligatoire** (erreur de compilation sinon).

D. Sortie en cours de répétition et usage de break (ES.77)

- L'instruction break n'est pas un réflexe paresseux mais une nécessité lorsque le corps de boucle est scindé en deux traitements Q1 et Q2 avec condition d'arrêt intermédiaire.

3. Programme d'Application Complet : Validation & Somme Cumulative

Ce programme illustre la combinaison canonique à l'IUT d'Anglet : saisie sécurisée avec do...while puis traitement cumulatif déterminé avec for, dans le strict respect de la **norme de style Matéi de Saint Leger** (Cartouche officiel, tripartition, flèches de flot de données et formatage Allman).

```
/*
 * Programme : CalculSommeEntiersPositifs
 * But : Saisir un entier N strictement positif (> 0) avec validation par post-condition,
 *       puis calculer la somme des entiers de 1 a N via une boucle comptee.
 * Date de derniere modification : 2026-11-08
 * Auteur : Matei de Saint Leger
 * Remarques : Tripartition canonique, indentation Allman 4 espaces, validation robuste.
 */

#include <iostream>
using namespace std;

int main(void)
{
    // CONSTANTES
    const int BORNE_MIN = 1;     // Valeur minimale admise pour N

    // VARIABLES
    int n;                       // Entier superieur ou egal a 1 saisi par l'utilisateur
    int somme;                   // Somme cumulative calculee (1 + 2 + ... + N)
    int i;                       // Variable de controle de l'iteration

    // TRAITEMENTS
    // 1. Saisie securisee par post-condition (au moins une saisie requise)
    // (clavier) >> saisirEntierControle >> n
    do
    {
        cout << "Entrez un entier N strictement positif (N >= " << BORNE_MIN << ") : ";
        cin >> n;

        if (n < BORNE_MIN)
        {
            cout << "[ERREUR] Valeur invalide. Veuillez recommencer." << endl;
        }
    }
    while (n < BORNE_MIN);

    // 2. Initialisation de l'accumulateur
    somme = 0;

    // 3. Boucle comptee pour le calcul cumulatif
    // n, BORNE_MIN >> accumulerSomme >> somme
    for (i = BORNE_MIN; i <= n; i = i + 1)
    {
        somme = somme + i;
    }

    // 4. Restitution du resultat a l'utilisateur
    // somme, n >> afficherResultat >> (ecran)
    cout << "La somme des entiers de 1 a " << n << " est egale a : " << somme << endl;

    return 0;
}
```

4. Checklist Anti-Erreurs pour les Évaluations Machine (IUT Anglet)

■■ **Boucle infinie sur while :** Vérifiez que la variable de contrôle est systématiquement modifiée dans TOUTES les branches d'exécution du corps de boucle.

■■ **Point-virgule sur do...while :** La seule boucle C++ qui se termine par un point-virgule est le do { ... } while (cond);. Son omission provoque une erreur de syntaxe immédiate à la compilation.

■■ **Erreur 'off-by-one' (décalage d'un tour) :** Vérifiez attentivement si l'indice d'arrêt doit être < n ou <= n selon que l'on parcourt un tableau de 0 à n-1 ou que l'on compte de 1 à n.

■■ **Initialisation obligatoire de l'accumulateur :** Une variable de somme ou de produit DOIT être initialisée avant la boucle (ex: somme = 0 ou produit = 1). En C++, les variables non initialisées contiennent des valeurs résiduelles imprévisibles.